شرح سطر بسطر ١٦ — السلاسة (الحركة واللمس) وحذف الملف الشخصي

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

الشعور بأن التطبيق «سلس» لا يأتي من السرعة وحدها، بل من أن كل تغيير على الشاشة له حركة قصيرة تفسّره: شاشة تنتقل، عنصر يظهر، زر يستجيب للمس. هذه الملفات تضيف أربع طبقات: انتقال الشاشات (`page_transitions`)، استجابة الضغط (`pressable_scale` و`haptics`)، ظهور العناصر تباعًا (`fade_slide_in`).

القاعدة التي اتبعناها: الحركات قصيرة (90–320 ملّي ثانية)، وبمنحنى يبدأ سريعًا ويتباطأ (`easeOutCubic`)، وتُضاف عبر اختصارات (`.pressable()` و`.fadeIn(n)`) تُلحَق بنهاية الودجت فلا تغيّر بنية الكود ولا تضيف مستوى تداخل.

الحذف: `delete_profile_sheet.dart` ورقة من مرحلتين (رمز سري إن وُجد، ثم تأكيد يذكر ما سيُفقد). الحذف نفسه في `ProfileRepository.delete` ضمن معاملة واحدة، وتنفّذه شاشة حسابي بعد التأكيد.

lib/app/page_transitions.dart — حركة الانتقال بين الشاشات

lib/app/page_transitions.dart

الأسطر 1–2

```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material (للودجتات والحركة) وgo_router (لنوع `GoRouterState`).

لماذا بهذه الطريقة؟ الملف جزء من طبقة `app/` التي تعرف الراوتر.

الأسطر 4–12

```dart
/// نوع حركة الانتقال بين الشاشات.
enum PageMotion {
  /// شاشات التبويبات (الرئيسية، حسابي): تلاشٍ سريع بلا انزلاق، فلا يبدو
  /// التنقّل بينها كأنه دخول إلى شاشة أعمق.
  tab,

  /// شاشات تُفتح فوق غيرها: تلاشٍ مع انزلاق خفيف من جهة القراءة.
  push,
}
```

ماذا يفعل؟ تعداد `PageMotion` بقيمتين: `tab` و`push`، لكل منهما تعليق.

لماذا بهذه الطريقة؟ نصنّف الشاشات حسب «معناها»: التبويبات نقاط وصول متوازية فتنتقل بتلاشٍ هادئ، والباقي شاشات تُفتح فوق غيرها فتنزلق قليلًا لتوحي بالعمق. تعداد لا منطقي (bool) ليتّسع لأنواع جديدة.

الأسطر 14–19

```
/// صفحة بحركة انتقال مخصّصة بدل الحركة الافتراضية للنظام.
CustomTransitionPage<void> appPage({
  required GoRouterState state,
  required Widget child,
  PageMotion motion = PageMotion.push,
}) {
```

ماذا يفعل؟ دالة `appPage` تأخذ حالة الراوتر والشاشة والحركة (الافتراضي `push`) وتعيد `CustomTransitionPage`.

لماذا بهذه الطريقة؟ بديل `MaterialPage` الذي يستعمل حركة النظام الثقيلة (تكبير على أندرويد). `state` لازم للمفتاح.

الأسطر 20–21

```
  final isTab = motion == PageMotion.tab;
  final duration = Duration(milliseconds: isTab ? 180 : 260);
```

ماذا يفعل؟ نعرف إن كانت الحركة `tab` ونحدد المدة: 180 ملّي ثانية للتبويب و260 للدفع.

لماذا بهذه الطريقة؟ التبويب أسرع لأن المستخدم يتنقّل بينها كثيرًا؛ أكثر من 300 تبدو بطيئة وأقل من 150 تبدو فجائية.

الأسطر 23–27

```
  return CustomTransitionPage<void>(
    key: state.pageKey,
    child: child,
    transitionDuration: duration,
    reverseTransitionDuration: duration,
```

ماذا يفعل؟ نبني الصفحة: المفتاح `state.pageKey`، والشاشة، ومدة الدخول ومدة الخروج متساويتان.

لماذا بهذه الطريقة؟ المفتاح يمنع إعادة بناء الصفحة بلا داعٍ. تساوي المدتين يجعل الرجوع صورة معكوسة للدخول.

السطر 28

```
    transitionsBuilder: (context, animation, secondaryAnimation, child) {
```

ماذا يفعل؟ `transitionsBuilder` يستقبل الحركة والشاشة ويعيد الودجت المتحرّك.

لماذا بهذه الطريقة؟ يُستدعى عند كل إطار من الحركة.

الأسطر 29–33

```
      final curved = CurvedAnimation(
        parent: animation,
        curve: Curves.easeOutCubic,
        reverseCurve: Curves.easeInCubic,
      );
```

ماذا يفعل؟ `CurvedAnimation` بمنحنى `easeOutCubic` للدخول و`easeInCubic` للخروج.

لماذا بهذه الطريقة؟ الدخول يبدأ سريعًا ويتباطأ فيبدو الاستجابة فورية، والخروج يتسارع فيختفي بسرعة.

الأسطر 34–35

```
34        final faded = FadeTransition(opacity: curved, child: child);
35        if (isTab) return faded;
```

ماذا يفعل؟ تلاشي (`FadeTransition`)، وللتبويب نكتفي به.

لماذا بهذه الطريقة؟ بلا انزلاق فلا يوحي التبديل بين التبويبات بأننا دخلنا شاشة أعمق.

الأسطر 37–38

```
37        // «في» نحو الأمام» وفي RTL يعني من اليسار، وفي LTR من اليمين.
38        final rtl = Directionality.of(context) == TextDirection.rtl;
```

ماذا يفعل؟ تعليق واتجاه القراءة: `rtl` من `Directionality`.

لماذا بهذه الطريقة؟ في RTL «الأمام» من اليسار، وفي LTR من اليمين. نقرأ الاتجاه بدل افتراض العربية فلا يتعطّل لو أُضيفت لغة أخرى.

الأسطر 39–45

```
39        return SlideTransition(
40          position: Tween<Offset>(
41            begin: Offset(rtl ? -0.06 : 0.06, 0),
42            end: Offset.zero,
43          ).animate(curved),
44          child: faded,
45        );
```

ماذا يفعل؟ `SlideTransition` بإزاحة 6% من عرض الشاشة (`±0.06`) نحو الصفر، والشاشة نفسها ملفوفة بالتلاشي.

لماذا بهذه الطريقة؟ إزاحة صغيرة تكفي للإيحاء بالاتجاه دون أن تُشعر بحركة كبيرة تثقل العين. القيمة بنسبة من العرض فتصلح لكل الأحجام.

الأسطر 46–48

```
46      },
47    );
48  }
```

ماذا يفعل؟ إغلاق الدوال.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/pressable_scale.dart — استجابة الضغط

lib/core/widgets/pressable_scale.dart

السطر 1

```
1   import 'package:flutter/material.dart';
```

ماذا يفعل؟ استيراد Material.

لماذا بهذه الطريقة؟ للودجتات وAnimatedScale.

الأسطر 3–5

```
/// «ضغطت» بأن اللمسة تم يعيده، فيشعر الطالب بأن قليلًا أثناء الضغط «يصغّر عنصرَه
/// أي مع فيعمل ،(فقط يستمع) فيها يتدخل ولا اللمسة يمنع لا .شيئًا `InkWell`.
class PressableScale extends StatefulWidget {
```

ماذا يفعل؟ تعليق وصنف `PressableScale` من نوع `StatefulWidget`.

لماذا بهذه الطريقة؟ التعليق يوضح أنه «يستمع فقط» ولا يعترض اللمسة. الحالة (هل الإصبع ضاغط) محلية فهو Stateful.

الأسطر 6–11

```
  const PressableScale({
    super.key,
    required this.child,
    this.scale = 0.97,
    this.enabled = true,
  });
```

ماذا يفعل؟ الباني: الودجت الابن والمقياس (0.97) وهل هو مفعّل.

لماذا بهذه الطريقة؟ 0.97 تصغير 3% يُحَسّ ولا يُرى كقفزة. `enabled` ليتعطل مع الأزرار المعطّلة.

الأسطر 13–19

```
  final Widget child;

  /// مقياس الحجم أثناء الضغط. (1 = بلا تغيير)
  final double scale;

  /// لا يتغيّر الحجم. false (زر معطّل) إن كان
  final bool enabled;
```

ماذا يفعل؟ الحقول وتعليقاتها.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 21–23

```
  @override
  State<PressableScale> createState() => _PressableScaleState();
}
```

ماذا يفعل؟ `createState` وإغلاق الصنف.

لماذا بهذه الطريقة؟ قالب `StatefulWidget`.

الأسطر 25–26

```
class _PressableScaleState extends State<PressableScale> {
  bool _down = false;
```

ماذا يفعل؟ حالة `_down` ابتداؤها false.

لماذا بهذه الطريقة؟ يتغيّر مع نزول الإصبع ورفعه.

الأسطر 28–30

```
28    void _set(bool value) {
29      if (_down != value) setState(() => _down = value);
30    }
```

ماذا يفعل؟ `_set` يغيّر الحالة فقط إن اختلفت القيمة.

لماذا بهذه الطريقة؟ يتجنب إعادة بناء بلا تغيير (أحداث نزول متكررة).

الأسطر 32–33

```
32    @override
33    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build`.

لماذا بهذه الطريقة؟ يعيد الودجت المتحرّك.

الأسطر 34–37

```
34      return Listener(
35        onPointerDown: (_) => _set(true),
36        onPointerUp: (_) => _set(false),
37        onPointerCancel: (_) => _set(false),
```

ماذا يفعل؟ `Listener` يستمع لنزول الإصبع ورفعه وإلغاء اللمسة.

لماذا بهذه الطريقة؟ `Listener` يتلقى أحداث المؤشّر الخام دون أن يدخل في «مسابقة الإيماءات»، فلا يسرق اللمسة من `InkWell` ولا `GestureDetector` بداخله. الإلغاء مهم: لو سحب الإصبع خارج الزر لا يبقى مصغّرًا.

الأسطر 38–43

```
38        child: AnimatedScale(
39          scale: _down && widget.enabled ? widget.scale : 1,
40          duration: const Duration(milliseconds: 90),
41          curve: Curves.easeOut,
42          child: widget.child,
43        ),
```

ماذا يفعل؟ `AnimatedScale` بمقياس 0.97 أثناء الضغط (إن كان مفعّلًا) وإلا 1، لمدة 90 ملّي ثانية بمنحنى `easeOut`.

لماذا بهذه الطريقة؟ ودجت جاهز للتحريك الضمني فلا نكتب `AnimationController`. 90 ملّي ثانية سريعة بما يكفي لتبدو لحظية.

الأسطر 44–46

```
44      );
45    }
46  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 48–53

```
/// اختصار: `widget.pressable()` يلفّ الودجت بـ [PressableScale] دون إعادة ترتيب الكود (لا مستوى تداخل إضافي).
extension PressableWidget on Widget {
  Widget pressable({bool enabled = true, double scale = 0.97}) =>
      PressableScale(enabled: enabled, scale: scale, child: this);
}
```

ماذا يفعل؟ امتداد `PressableWidget` على `Widget`: الدالة `pressable({enabled, scale})`.

لماذا بهذه الطريقة؟ اختصار يُلحَق بنهاية الودجت (`...).pressable()`) فلا نضيف مستوى تداخل ولا نعيد ترتيب كود الشاشات الموجودة. هذا ما جعل التعديل صغيرًا في الأزرار والبطاقات والتبويبات.

lib/core/widgets/fade_slide_in.dart — ظهور العناصر تباعًا

lib/core/widgets/fade_slide_in.dart

السطر 1

```
import 'package:flutter/material.dart';
```

ماذا يفعل؟ استيراد Material.

لماذا بهذه الطريقة؟ للودجتات والحركة.

الأسطر 3–7

```
/// يؤخّر البداية [delay]. يُظهر عنصره بتلاشٍ وانزلاق خفيف للأعلى عند أول ظهور
/// فيظهر عناصر القائمة تباعًا بدل أن تقفز دفعة واحدة.
///
/// لا مؤقّت منفصل: لا يترك مؤقّتات معلّقة (`Interval`) التأخير داخل منحنى واحد
/// في الاختبارات، ويتوقف فورًا إن أُزيلت الشاشة.
```

ماذا يفعل؟ تعليق: تلاشٍ وانزلاق خفيف للأعلى، والتأخير داخل منحنى واحد لا مؤقّت.

لماذا بهذه الطريقة؟ سبب اختيار `Interval`: لو استعملنا `Future.delayed` لبقيت مؤقّتات معلّقة في الاختبارات ولتأخرت بعد إزالة الشاشة.

الأسطر 8–15

```
class FadeSlideIn extends StatefulWidget {
  const FadeSlideIn({
    super.key,
    required this.child,
    this.delay = Duration.zero,
    this.duration = const Duration(milliseconds: 320),
    this.offset = 14,
  });
```

ماذا يفعل؟ صنف `FadeSlideIn` وباني: الابن والتأخير (صفر) والمدة (320) والإزاحة (14 بكسلًا).

لماذا بهذه الطريقة؟ 320 ملّي ثانية بحركة ناعمة، و14 بكسلًا انزلاق يكفي لإحساس «الطفو» دون مبالغة.

الأسطر 17–22

```
  final Widget child;
  final Duration delay;
  final Duration duration;

  /// مسافة الانزلاق الابتدائية بالبكسل (من الأسفل).
  final double offset;
```

ماذا يفعل؟ الحقول وتعليق الإزاحة.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 24–26

```
  @override
  State<FadeSlideIn> createState() => _FadeSlideInState();
}
```

ماذا يفعل؟ `createState`.

لماذا بهذه الطريقة؟ قالب `StatefulWidget`.

الأسطر 28–31

```
class _FadeSlideInState extends State<FadeSlideIn>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _curve;
```

ماذا يفعل؟ حالة `_FadeSlideInState` بمزيج `SingleTickerProviderStateMixin`، وحقلا المتحكم والمنحنى.

لماذا بهذه الطريقة؟ المزيج يعطي `vsync` للمتحكم ليتزامن مع إطارات الشاشة.

الأسطر 33–37

```
  @override
  void initState() {
    super.initState();
    final total = widget.delay + widget.duration;
    _controller = AnimationController(vsync: this, duration: total);
```

ماذا يفعل؟ `initState`: المدة الكلية = التأخير + المدة، وننشئ المتحكم بها.

لماذا بهذه الطريقة؟ نحسب الكل كمدة واحدة ثم نقتطع جزء التأخير بالمنحنى.

الأسطر 38–45

```
38      _curve = CurvedAnimation(
39        parent: _controller,
40        curve: Interval(
41          widget.delay.inMilliseconds / total.inMilliseconds,
42          1,
43          curve: Curves.easeOutCubic,
44        ),
45      );
```

ماذا يفعل؟ `CurvedAnimation` بـ `Interval(بداية = التأخير ÷ الكل، نهاية = 1)` ومنحنى `easeOutCubic`.

لماذا بهذه الطريقة؟ قبل `begin` تبقى القيمة 0 (شفاف)، وبعده يجري الدخول. هكذا يتأخر العنصر دون أي مؤقّت.

الأسطر 46–47

```
46      _controller.forward();
47    }
```

ماذا يفعل؟ نشغّل المتحكم مرة واحدة ثم إغلاق `initState`.

لماذا بهذه الطريقة؟ الظهور يحدث عند أول بناء فقط، لا عند كل إعادة بناء.

الأسطر 49–53

```
49    @override
50    void dispose() {
51      _controller.dispose();
52      super.dispose();
53    }
```

ماذا يفعل؟ `dispose` يتخلّص من المتحكم.

لماذا بهذه الطريقة؟ منع تسرّب الذاكرة وتوقف الحركة إن غادر الطالب الشاشة.

الأسطر 55–59

```
55    @override
56    Widget build(BuildContext context) {
57      return AnimatedBuilder(
58        animation: _curve,
59        child: widget.child,
```

ماذا يفعل؟ `AnimatedBuilder` : `build` يعيد البناء مع كل إطار، والابن يُمرَّر `child` لئلا يُعاد بناؤه.

لماذا بهذه الطريقة؟ الابن (الشاشة الثقيلة) يُبنى مرة واحدة، ويتغيّر فقط التلاشي والإزاحة حوله: أداء أفضل.

الأسطر 60–66

```
60        builder: (context, child) => Opacity(
61          opacity: _curve.value,
62          child: Transform.translate(
63            offset: Offset(0, (1 - _curve.value) * widget.offset),
64            child: child,
65          ),
66        ),
```

ماذا يفعل؟ `Opacity` بقيمة المنحنى و`Transform.translate` بإزاحة `الإزاحة × (القيمة − 1)` رأسيًا.

لماذا بهذه الطريقة؟ عند 0: شفاف ومزاح 14 للأسفل، وعند 1: ظاهر في مكانه. إزاحة التحويل لا تؤثر في التخطيط فلا تتحرك العناصر المجاورة.

الأسطر 67–69

```
67      );
68    }
69  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 71–78

```
71  /// يُظهر الودجت بتلاشٍ وانزلاق بعد `45 × 2` ملّي ثانية، اختصار: `widget.fadeIn(2)`.
72  /// فتظهر عناصر الصفحة تباعًا. لا يضيف مستوى تداخل في الكود.
73  extension FadeInWidget on Widget {
74    Widget fadeIn([int order = 0]) => FadeSlideIn(
75      delay: Duration(milliseconds: 45 * order),
76      child: this,
77    );
78  }
```

ماذا يفعل؟ امتداد `FadeInWidget` على `Widget`: `fadeIn([order])` يعيد `FadeSlideIn` بتأخير `45 × order` ملّي ثانية.

لماذا بهذه الطريقة؟ اختصار بترتيب العنصر في الصفحة: الأول فورًا، والثاني بعد 45، وهكذا، فيظهر المحتوى تباعًا من الأعلى للأسفل كموجة قصيرة بدل قفزة واحدة.

lib/core/utils/haptics.dart — اهتزازات اللمس

`lib/core/utils/haptics.dart`

السطر 1

```
1   import 'package:flutter/services.dart';
```

ماذا يفعل؟ استيراد `services` (لـ `HapticFeedback`).

لماذا بهذه الطريقة؟ واجهة النظام للاهتزاز.

الأسطر 3–4

```
/// اهتزازات خفيفة تؤكد اللمس. مكان واحد فيسهل تعطيلها أو تعديلها لاحقًا.
abstract final class Haptics {
```

ماذا يفعل؟ تعليق وصنف `Haptics` ثابت (`abstract final`).

لماذا بهذه الطريقة؟ مكان واحد للاهتزازات فنعدّلها أو نعطّلها كلها من هنا لاحقًا (مثلًا إعداد «اهتزاز اللمس»).

الأسطر 5–6

```
  /// نقرة خفيفة: مفاتيح، أرقام، تبويبات، اختيارات.
  static void tap() => HapticFeedback.selectionClick();
```

ماذا يفعل؟ `tap()` : `selectionClick` — نقرة خفيفة.

لماذا بهذه الطريقة؟ للمفاتيح والأرقام والتبويبات والاختيارات: تأكيد دقيق لا يزعج.

الأسطر 8–10

```
  /// اهتزاز أقوى لإجراء لا رجوع عنه (حذف).
  static void heavy() => HapticFeedback.mediumImpact();
}
```

ماذا يفعل؟ `heavy()` : `mediumImpact` — اهتزاز أقوى، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ لإجراء لا رجوع عنه (تأكيد الحذف) كي يشعر الطالب بثقله.

lib/features/account/widgets/delete_profile_sheet.dart — ورقة حذف الملف

`lib/features/account/widgets/delete_profile_sheet.dart`

السطر 1

```
import 'dart:async';
```

ماذا يفعل؟ استيراد `dart:async` (لـ `Timer`).

لماذا بهذه الطريقة؟ لعدّاد القفل التنازلي.

الأسطر 3–15

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../core/db/app_database.dart';
import '../../../core/strings/ar.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_text_styles.dart';
import '../../../core/utils/pin_hasher.dart';
import '../../../core/widgets/pin_pad.dart';
import '../../../core/widgets/primary_button.dart';
import '../../profiles/profile_providers.dart';
import '../../profiles/profile_repository.dart';
import '../../../core/utils/haptics.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod والقاعدة والنصوص والألوان والأنماط وأداة الرمز السري ولوحة الأرقام والزر ومزوّدات الملفات والمستودع والاهتزازات.

لماذا بهذه الطريقة؟ الورقة تعيد استعمال لوحة الأرقام والتحقق من الرمز الموجودين فلا نكرر منطقًا.

الأسطر 17–23

```
/// ورقة تأكيد حذف الملف الشخصي. إن كان للملف رمز سري يُطلب أولًا (حتى لا
/// يحذف أحد ملف أخيه)، ثم شاشة تأكيد تذكر ما سيُحذف.
/// تعيد true فقط إن أكّد الطالب؛ الحذف نفسه تنفّذه الشاشة المستدعية.
Future<bool> showDeleteProfileSheet(
  BuildContext context,
  Profile profile,
) async {
```

ماذا يفعل؟ تعليق ودالة `showDeleteProfileSheet(context, profile)` تعيد `Future<bool>`.

لماذا بهذه الطريقة؟ التعليق يشرح القصد: الرمز السري أولًا حتى لا يحذف أحد ملف أخيه. تعيد true فقط عند التأكيد، والحذف الفعلي يبقى لشاشة حسابي (فصل بين السؤال والتنفيذ، وأسهل اختبارًا).

الأسطر 24–32

```
  final confirmed = await showModalBottomSheet<bool>(
    context: context,
    isScrollControlled: true,
    backgroundColor: AppColors.ground,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
    ),
    builder: (_) => _DeleteProfileSheet(profile: profile),
  );
```

ماذا يفعل؟ نفتح الورقة (قابلة للتمرير، خلفية `ground`، زوايا 28) وبناؤها `_DeleteProfileSheet`.

لماذا بهذه الطريقة؟ نفس شكل أوراق حسابي الأخرى.

الأسطر 33–34

```
  return confirmed ?? false;
}
```

ماذا يفعل؟ إغلاق الورقة بلا اختيار (سحب أو لمس خارجها) يعني `null` فنعيد false.

لماذا بهذه الطريقة؟ `?? false` الأكثر أمانًا: أي إغلاق غامض لا يحذف.

الأسطر 36–44

```
class _DeleteProfileSheet extends ConsumerStatefulWidget {
  const _DeleteProfileSheet({required this.profile});

  final Profile profile;

  @override
  ConsumerState<_DeleteProfileSheet> createState() =>
      _DeleteProfileSheetState();
}
```

ماذا يفعل؟ صنف `_DeleteProfileSheet` من نوع `ConsumerStatefulWidget` وحقله الملف.

لماذا بهذه الطريقة؟ يحتاج `ref` (للمستودع) وحالة محلية (الخطوة والإدخال).

الأسطر 46–47

```
class _DeleteProfileSheetState extends ConsumerState<_DeleteProfileSheet> {
  late bool _needsPin = widget.profile.pinHash != null;
```

ماذا يفعل؟ حالة الورقة: هل نحتاج الرمز (`pinHash != null`).

لماذا بهذه الطريقة؟ `late` تُحسب مرة واحدة من الملف. تنتقل إلى false بعد التحقق الناجح.

الأسطر 48–52

```
  String _entry = '';
  bool _wrong = false;
  bool _busy = false;
  int _lockedSeconds = 0;
  Timer? _ticker;
```

ماذا يفعل؟ حقول: الإدخال والخطأ و«مشغول» وثواني القفل والمؤقّت.

لماذا بهذه الطريقة؟ نفس حقول شاشة إدخال الرمز.

الأسطر 54–58

```
  @override
  void dispose() {
    _ticker?.cancel();
    super.dispose();
  }
```

ماذا يفعل؟ `dispose` يلغي المؤقّت.

لماذا بهذه الطريقة؟ يمنع استدعاء `setState` بعد إغلاق الورقة.

الأسطر 60–63

```
60    void _onDigit(String digit) {
61      if (_busy || _lockedSeconds > 0 || _entry.length >= PinHasher.pinLength) {
62        return;
63      }
```

ماذا يفعل؟ `_onDigit`: نتجاهل الإدخال إن كنا مشغولين أو مقفولين أو اكتملت الأرقام.

لماذا بهذه الطريقة؟ يمنع إدخالًا أثناء التحقق أو فترة القفل.

الأسطر 64–69

```
64      setState(() {
65        _wrong = false;
66        _entry += digit;
67      });
68      if (_entry.length == PinHasher.pinLength) _verify();
69    }
```

ماذا يفعل؟ نضيف الرقم ونزيل علامة الخطأ، وعند اكتمال الأرقام نتحقق.

لماذا بهذه الطريقة؟ التحقق تلقائي بلا زر.

الأسطر 71–74

```
71    void _onDelete() {
72      if (_busy || _entry.isEmpty) return;
73      setState(() => _entry = _entry.substring(0, _entry.length - 1));
74    }
```

ماذا يفعل؟ `_onDelete`: يحذف آخر رقم إن وُجد وغير مشغول.

لماذا بهذه الطريقة؟ لوحة الأرقام لها زر حذف.

الأسطر 76–79

```
76    Future<void> _verify() async {
77      setState(() => _busy = true);
78      // وقفة قصيرة ليظهر امتلاء النقطة الرابعة قبل النتيجة.
79      await Future<void>.delayed(const Duration(milliseconds: 150));
```

ماذا يفعل؟ `_verify`: نصبح «مشغولين» ثم وقفة 150 ملّي ثانية.

لماذا بهذه الطريقة؟ تظهر النقطة الرابعة ممتلئة قبل النتيجة كما في بقية الشاشات.

الأسطر 80–83

```
80      final result = await ref
81          .read(profileRepositoryProvider)
82          .verifyPin(widget.profile.id, _entry);
83      if (!mounted) return;
```

ماذا يفعل؟ نتحقق عبر `verifyPin` (يطبّق قفل الـ 5 محاولات نفسه) ونتأكد أن الورقة ما زالت مفتوحة.

لماذا بهذه الطريقة؟ نفس المستودع فلا يمكن تجاوز القفل بالحذف.

الأسطر 85–90

```
85      switch (result.check) {
86        case PinCheck.ok:
87          setState(() {
88            _needsPin = false;
89            _busy = false;
90          });
```

ماذا يفعل؟ الرمز صحيح: `_needsPin = false` فتظهر خطوة التأكيد.

لماذا بهذه الطريقة؟ الانتقال بين الخطوتين بتبديل الودجت المعروض.

الأسطر 91–96

```
91        case PinCheck.wrong:
92          setState(() {
93            _wrong = true;
94            _entry = '';
95            _busy = false;
96          });
```

ماذا يفعل؟ رمز خاطئ: علامة خطأ ومسح الإدخال.

لماذا بهذه الطريقة؟ يعيد المحاولة.

الأسطر 97–105

```
97        case PinCheck.locked:
98          _startCountdown(result.lockedFor ?? const Duration(seconds: 30));
99          setState(() {
100           _wrong = false;
101           _entry = '';
102           _busy = false;
103         });
104     }
105   }
```

ماذا يفعل؟ مقفول: نبدأ العدّ التنازلي ونمسح الإدخال.

لماذا بهذه الطريقة؟ المدة من المستودع (الافتراضي 30 ثانية).

الأسطر 107–115

```
107   void _startCountdown(Duration duration) {
108     _ticker?.cancel();
109     _lockedSeconds = (duration.inMilliseconds / 1000).ceil();
110     _ticker = Timer.periodic(const Duration(seconds: 1), (timer) {
111       if (!mounted) return;
112       setState(() => _lockedSeconds = (_lockedSeconds - 1).clamp(0, 9999));
113       if (_lockedSeconds == 0) timer.cancel();
114     });
115   }
```

ماذا يفعل؟ `_startCountdown`: يحوّل المدة إلى ثوانٍ (تقريب لأعلى) ويشغّل مؤقّتًا يتناقص كل ثانية ويتوقف عند الصفر.

لماذا بهذه الطريقة؟ `ceil` فلا تظهر «0 ثانية» والزر ما زال مقفولًا. `clamp` يمنع قيمة سالبة.

الأسطر 117–128

```
117   @override
118   Widget build(BuildContext context) {
119     return SafeArea(
120       child: SingleChildScrollView(
121         padding: const EdgeInsetsDirectional.fromSTEB(24, 24, 24, 16),
122         child: AnimatedSwitcher(
123           duration: const Duration(milliseconds: 200),
124           child: _needsPin ? _pinStage() : _confirmStage(),
125         ),
126       ),
127     );
128   }
```

ماذا يفعل؟ `SafeArea` : `build` ثم `SingleChildScrollView` ثم `AnimatedSwitcher` بمدة 200 ملّي ثانية يعرض خطوة الرمز أو التأكيد.

لماذا بهذه الطريقة؟ `AnimatedSwitcher` يلاشي بين الخطوتين بدل قفزة. التمرير لحماية الشاشات القصيرة والخط المكبّر.

الأسطر 130–140

```
130   Widget _pinStage() {
131     return Column(
132       key: const ValueKey('pin'),
133       mainAxisSize: MainAxisSize.min,
134       spacing: 14,
135       children: [
136         Text(
137           ArStrings.deletePinTitle,
138           textAlign: TextAlign.center,
139           style: AppTextStyles.section18,
140         ),
```

ماذا يفعل؟ خطوة الرمز: عمود بمفتاح `ValueKey("pin")` وعنوان «أدخل رمزك السري لتأكيد الحذف».

لماذا بهذه الطريقة؟ المفتاح ضروري كي يعرف `AnimatedSwitcher` أن الودجت تغيّر.

الأسطر 141–145

```
141         PinDots(
142           length: PinHasher.pinLength,
143           filled: _entry.length,
144           error: _wrong,
145         ),
```

ماذا يفعل؟ نقاط الرمز تعكس عدد الأرقام وحالة الخطأ.

لماذا بهذه الطريقة؟ نفس `PinDots` المشترك.

الأسطر 146–153

```
        SizedBox(
          height: 24,
          child: _lockedSeconds > 0
              ? _message(ArStrings.pinLocked(_lockedSeconds))
              : _wrong
              ? _message(ArStrings.pinWrong)
              : null,
        ),
```

ماذا يفعل؟ سطر رسالة بارتفاع ثابت 24: عدّاد القفل أو «الرمز غير صحيح» أو فراغ.

لماذا بهذه الطريقة؟ الارتفاع الثابت يمنع قفز اللوحة عند ظهور الرسالة.

الأسطر 154–161

```
        PinPad(
          onDigit: _onDigit,
          onDelete: _onDelete,
          enabled: !_busy && _lockedSeconds == 0,
        ),
      ],
    );
  }
```

ماذا يفعل؟ لوحة الأرقام، معطّلة أثناء التحقق أو القفل.

لماذا بهذه الطريقة؟ `enabled` تُوقف الضغط والاهتزاز معًا.

الأسطر 163–169

```
  Widget _message(String text) {
    return Text(
      text,
      textAlign: TextAlign.center,
      style: AppTextStyles.label14.copyWith(color: AppColors.coral),
    );
  }
```

ماذا يفعل؟ `_message`: نص مرجاني بنمط `label14`.

لماذا بهذه الطريقة؟ دالة صغيرة تمنع تكرار الأسلوب مرتين.

الأسطر 171–177

```
  Widget _confirmStage() {
    return Column(
      key: const ValueKey('confirm'),
      mainAxisSize: MainAxisSize.min,
      crossAxisAlignment: CrossAxisAlignment.stretch,
      spacing: 12,
      children: [
```

ماذا يفعل؟ خطوة التأكيد: عمود بمفتاح `ValueKey("confirm")` ممتد بمسافة 12.

لماذا بهذه الطريقة؟ مفتاح مختلف عن الأول.

الأسطر 178–182

```
        Text(
          ArStrings.deleteTitle(widget.profile.name),
          style: AppTextStyles.section18,
        ),
        Text(ArStrings.deleteWarning, style: AppTextStyles.body15),
```

ماذا يفعل؟ العنوان «حذف ملف «الاسم»؟» ثم تحذير «لا يمكن التراجع».

لماذا بهذه الطريقة؟ ذكر الاسم يؤكد أي ملف يُحذف.

الأسطر 183–185

```
        const _LossItem(ArStrings.deleteItem1),
        const _LossItem(ArStrings.deleteItem2),
        const _LossItem(ArStrings.deleteItem3),
```

ماذا يفعل؟ ثلاثة عناصر `_LossItem` تقول ما سيُفقد.

لماذا بهذه الطريقة؟ الشفافية: يعرف الطالب بالضبط ما يخسره.

السطر 186

```
        const SizedBox(height: 4),
```

ماذا يفعل؟ مسافة 4.

لماذا بهذه الطريقة؟ تفصل التحذير عن الزر.

الأسطر 187–194

```
        PrimaryButton(
          label: ArStrings.deleteConfirm,
          backgroundColor: AppColors.coral,
          onPressed: () {
            Haptics.heavy();
            Navigator.of(context).pop(true);
          },
        ),
```

ماذا يفعل؟ زر «احذف نهائيًا» مرجاني: اهتزاز قوي ثم إغلاق الورقة بـ true.

لماذا بهذه الطريقة؟ اللون المرجاني تحذير. الاهتزاز يعطي إحساسًا بثقل الفعل.

الأسطر 195–199

```
        TextButton(
          onPressed: () => Navigator.of(context).pop(false),
          child: const Text(ArStrings.deleteCancel),
        ),
      ],
```

ماذا يفعل؟ زر «تراجع» نصي يغلق بـ false، ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ التراجع بنفس مستوى السهولة، وأقل بروزًا من الحذف.

الأسطر 200–202

```
    );
  }
}
```

ماذا يفعل؟ إغلاق الدوال والصنف.

لماذا بهذه الطريقة؟ نهاية الحالة.

الأسطر 204–208

```
class _LossItem extends StatelessWidget {
  const _LossItem(this.text);

  final String text;
```

ماذا يفعل؟ صنف `_LossItem` بحقل النص.

لماذا بهذه الطريقة؟ سطر واحد في قائمة الفقدان.

الأسطر 209–219

```
  @override
  Widget build(BuildContext context) {
    return Row(
      spacing: 10,
      children: [
        const Icon(Icons.close_rounded, size: 20, color: AppColors.coral),
        Expanded(child: Text(text, style: AppTextStyles.label14)),
      ],
    );
  }
}
```

ماذا يفعل؟ `Row` : `build` بأيقونة × مرجانية ونص `label14` في `Expanded` ، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ × توحي بالفقدان، و `Expanded` يمنع الفيض بالنصوص الطويلة.